

ネットワーク及び演習

第 1 回 イントロと Python 入門

佐野 彰

本日の内容

1. 科目の概要 (20 分)

- 到達目標、実施方法、成績評価、質問方法

2. Python の実行環境 (15 分)

- インタプリタ、対話モード、スクリプト実行、ノートブック

3. Python の基本 — C 言語との対比で (30 分)

- 動的型付け、インデント構文、変数と型、制御構文、リスト

4. 演習課題の進め方 (5 分)

- paiza ラーニング、aiJudge (オンラインジャッジ)

1. 科目の概要

この科目の到達目標

Network（を理解する）

- 計算機ネットワークの**基本概念**を説明できる
 - クライアント／サーバ、IP アドレスとポート、TCP、HTTP、スレッド
- ネットワークを介して通信する**プログラムの仕組み**を、実際に動かして理解する

Python（を使って）

- Python を利用した**簡単なプログラミング**ができる
 - 変数・制御構文・関数・リスト・文字列
 - `socket` , `threading` などの標準ライブラリ
- 「動くコードを自分で書いて確かめる」習慣をつける

****Python を使って、Network を理解する** 科目です。Python 自体を深く学ぶ科目ではありません。**

- 教職課程（情報）の必修科目でもある

なぜ Python なのか？ — 同じ「echo クライアント」を C と比べる

```
# Python: 11 行
import socket

HOST = 'localhost' # サーバのホスト名
PORT = 50007       # サーバのポート番号

with socket.socket(socket.AF_INET,
                   socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
    print('Received', repr(data))
```

```
/* C: 40 行超 (一部省略) */
#include <sys/socket.h>
#include <arpa/inet.h>
/* ... */
int main() {
    int sock;
    struct sockaddr_in addr;
    if ((sock = socket(PF_INET, SOCK_STREAM,
                      IPPROTO_TCP)) < 0) exit(1);
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(50007);
    if (connect(sock, (struct sockaddr *)&addr,
                sizeof(addr)) < 0) exit(2);
    /* send / recv / close ... */
}
```

- Python の方が圧倒的に簡潔 → ネットワークの仕組みそのものに集中できる
- C 固有の構造体・バイト順・メモリ管理に煩わされない

科目の位置づけ — 上下 1 階層分の視野を持つ

レベル	内容	対応する科目
1. 基盤	計算機の仕組み（ハードウェア、CPU、OS）	情報処理システムⅠ・Ⅱ
2. コーディング	言語の構文、変数・関数・制御構造	プログラミングⅠ・Ⅱ・Ⅲ
3. 計算的思考	アルゴリズム、データ構造	アルゴリズム・データ構造
4. ソフトウェア設計	部品の組み合わせ、環境構築、開発ツール	ネットワーク、オブジェクト指向、グラフィックス
5. ユーザー・社会	UI/UX、倫理的・持続的な意義	—

- この科目はレベル 2 → 4 への橋渡し。
「言語が書ける」から「通信する部品を組み合わせで動かせる」へ
- 生成 AI はレベル 2～3 を肩代わりしつつある。
だからこそ上下を見渡して設計できる力が問われる

科目の実施方法

毎回の流れ（金曜 4・5 講時）

- 4 講時: 講義パート（約1時間、Teams でリアルタイム配信・録画あり）
- 講義パート終了後： 前回内容の小テスト（10分）
- 5 講時: 実習（TA対応）

課題の種類

- paiza ラーニングの指定講座を受講
- aiJudge（オンラインジャッジ）でプログラミング課題を提出・自動採点
- 受講修了証などのスクリーンショットも aiJudgeに提出

提出期限

- 講義日を 1 日目とした 1 週間 — 翌週の金曜 0:00（= 木曜 24:00）締切

受講方法

- 4・5 講時とも **1-608 実習室**を対面で利用できる
- 講義パートは Teams 会議で配信、録画も閲覧可
- 完全オンラインでの受講も可能
- **TA のサポートが必要なら 1-608 で対面受講**
 - 対面を優先するため、オンラインでの TA サポートは保証しない

質問方法

- 科目内容の質問は Teams 「★ 質問用チャンネル」 へ
 - 他の受講者にも役立つ。時間外も歓迎（即答は期待しない）
- チャットやメールでの個別質問には原則応じない
- 演習時間中の質問は TA に直接
- 良い質問: 参加形態・席番号、環境、やったこと、エラーメッセージ
- 悪い質問: 「課題 1-3 ができません」

成績評価

配点

項目	配点
演習課題（ex1～ex7）	60 点
小テスト	40 点
└ 各回の小テスト（5 回予定）	各 1 点・計 5 点
└ 最終回のまとめの小テスト（11/13）	35 点（各回小テスト3回以上で受験可）
任意提出レポート	加点（詳細は後日）

生成 AI の利用

- 演習・デバッグでの利用は**推奨**（コードの解説、エラーの修正、24 時間対応）
- ただし **小テストでの利用は禁止**（Web 検索は可）。利用が判明した場合は不合格
- 「自分でPythonの簡単なコードを書ける」ことがこの科目の単位取得の条件

本日の目標

- Python の実行環境を構築する（事前課題 ex0 — まだの人は今日中に）
- 対話モード・スクリプト実行・ノートブックの **3 つの実行方法**を使い分けられる
- paiza ラーニングを使えるようになる
- aiJudge にログインして課題を提出できるようになる
- Python の特徴を **C との対比**で説明できる
 - インタプリタ、動的型付け、インデント構文
- Python の基本を理解する
 - 変数、基本型、`print` / `input`、`if`、`for` / `range`、`while`、リスト

2. Python の実行環境

インタプリタ、対話モード、スクリプト、ノートブック

コンパイラ言語とインタプリタ言語

C — コンパイラ言語



- 実行前に機械語に翻訳する
- 翻訳済みなので実行は速い
- 型のエラーはコンパイル時に見つかる

Python — インタプリタ言語



- コンパイル不要、書いてすぐ動かせる
- 実行は C より遅い（この科目では問題にならない）
- エラーは実行してその行に到達したときに
出る
 - → 動かして初めて発覚することがある

Python の 3 つの実行方法

① 対話モード (REPL)

```
$ python
>>> 1 + 2
3
>>> name = 'sano'
>>> print('Hello', name)
Hello sano
>>> exit()
```

- 1 行ずつ入力して即実行
- ちょっと試すのに最適
- `>>>` がプロンプト

② スクリプト実行

hello.py

```
name = 'sano'
print('Hello', name)
```

```
$ python hello.py
Hello sano
```

- ファイルに書いてまとめて実行
- 課題の提出はこの形式
- aiJudge もこの形で採点する

③ ノートブック

- ブラウザ上で「セル」ごとに実行
- 説明文・数式・コード・結果を一体化
- 教材はこの形式 (marimo)
- 科目ページから開けばインストール不要
- 中身はふつうの `.py` ファイル

macOS では `python3`、Windows では `python` または `py`。`python -V` でバージョン (3.x) を確認。

ノートブックの使い方 — セルと変数

- **セル**: ノートブックの最小単位。テキストセル (Markdown) とコードセル
- コードセルは `Shift + Enter` で実行。結果はセルの直下に表示
- セルは名前空間を共有する — あるセルで作った変数は他のセルで使える
- セル 1 の `a` を書き換えると、`a` を使うセル 2 は自動で再実行される

```
# セル 1  
a = 1234  
b = 5678
```

```
# セル 2  
b / a      # → 4.6013...
```

ノートブックの注意点

ルール: 同じ変数は 1 つのセルでしか定義できない

別のセルで `a = ...` と定義し直すとエラーになる。marimo は変数の依存関係を追跡して関係するセルを自動で再実行するため、「どのセルを何番目に実行したか」で結果が変わることはない。

- 変数を変えたいときは、定義しているセルを書き換える
- 教材ノートブックは科目ページからブラウザで開ける
 - `jupyter.py` (ノートブックの使い方)
 - `python_tutorial.py` (Python 入門)

3. Python の基本

C 言語との対比で

Python の 3 つの特徴 (C と比べて)

	C 言語	Python
① 実行方法	コンパイルして実行	インタプリタが直接実行 (→ 2 章)
② 変数の型	宣言が必要 (<code>int a;</code>)	動的型付け — 代入した値で決まる
③ ブロック構文	<code>{ }</code> で囲む (インデントは飾り)	インデントがブロックを表す

細かい違い (文末の `;` が不要、コメントは `#`、出力は `print()` など) は出てきたときに説明する。

ほかにもガベージコレクション、オブジェクト指向など違いは多いが、この科目ではこの 3 つを押さえれば十分。

特徴 ②: 動的型付け

```
a = 1          # a は int (整数)
x = 1.0        # x は float (浮動小数点)
s = 'abc'      # s は str (文字列)

print(type(a), type(x), type(s))
# <class 'int'> <class 'float'> <class 'str'>

a = 'hello'    # 同じ変数に別の型を代入できる
print(type(a))
# <class 'str'>
```

- 変数の宣言は不要。代入した瞬間に型が決まる。同じ変数に別の型の値も代入できる

```
a = 1
b = 2
print(a + b)      # 3
b = 'abc'
print(a + b)      # TypeError! int と str は足せない
print(str(a) + b) # '1abc' 型変換すれば OK
```

便利な反面、どの変数が今どの型かを自分で把握しておく必要がある。迷ったら `type()` で確認。

特徴 ③: インデントによるブロック構文

```
a = 10
if a > 0:
    print('a は正の数です')    # if のブロック
    print('処理を続けます')    # 同じブロック
print('ここは if の外')
```

- `:` の次の行から、同じ深さのインデントがブロック
- インデントは 半角スペース 4 つが標準（タブと混ぜない）

```
a = 10
if a > 0:
print('a は正の数です')    # IndentationError!
```

入れ子の深さと C との比較

入れ子は深さで表す:

```
for i in range(10):  
    if i % 2 == 0:  
        print(f'{i} は偶数')  
    else:  
        print(f'{i} は奇数')
```

同じものを C で書くと:

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        printf("%d は偶数\n", i);  
    } else {  
        printf("%d は奇数\n", i);  
    }  
}
```

変数と基本的な型

型	例	説明
int	42 , -3 , 10**100	整数。桁数の上限なし
float	3.14 , 1e-3	浮動小数点（C の double ）
str	'abc' , "abc"	文字列
bool	True , False	真偽値
bytes	b'Hello'	バイト列（通信で使う）

```
print(123 ** 99)    # C では溢れる大きさも扱える
print(7 / 2)        # 3.5   ← / は常に float
print(7 // 2)       # 3     ← 整数の商
print(7 % 2)        # 1     ← 余り
print(2 ** 10)      # 1024  ← べき乗
```

型変換と文字列の操作

型変換

```
n = int('123')      # str → int
s = str(45)          # int → str
f = float('2.5')     # str → float
```

文字列の操作

```
s = 'Hello' + ' ' + 'World'  # 連結
print(len(s))                # 11
print(s.upper())              # HELLO WORLD
print(s[0], s[-1])            # H d
print(s[0:5])                 # Hello
```

- 文字列は**変更不可** (immutable)。操作すると新しい文字列ができる

出力 — `print`

```
name = 'sano'
age = 20
print('Hello', name)           # Hello sano   (空白区切り)
print('Hello, ' + name + '!')  # Hello, sano!
print(f'{name} は {age} 歳')    # f 文字列 (推奨)
print(f'{3.14159:.2f}')         # 3.14
print('A', 'B', sep='-')       # A-B
print('no newline', end='')    # 改行しない
```

- C の `printf("%s は %d 歳\n", name, age)` に相当するのが f 文字列

入力 — `input`

```
s = input()          # 1 行読み込む（改行は除く）
n = int(input())      # 整数として読みたければ変換
a, b = input().split() # 空白区切りで 2 つ
x, y = map(int, input().split())
```

- `input()` の戻り値は常に `str`
- aiJudge の課題は「標準入力から読んで標準出力に書く」形式
 - `input()` と `print()` を組み合わせる

`input('名前: ')` のようにプロンプト文字列を渡せるが、ジャッジではプロンプトを出力しないこと（出力が一致しなくなる）。

条件分岐 — `if` / `elif` / `else`

```
n = int(input())
if n > 0:
    print('正')
elif n == 0:
    print('ゼロ')
else:
    print('負')

if 0 <= n < 10:          # 連鎖比較が書ける
    print('1桁')
if n % 2 == 0 and n > 0: # && → and, || → or, ! → not
    print('正の偶数')
```

- `else if` ではなく `elif`。条件に `( )` は不要、`:` を忘れない
- `0`, `0.0`, `''`, `[]`, `None` は偽、それ以外は真 (`if s:` など) で判定に使える)

C	<code>==</code> , <code>!=</code> , <code><</code> , <code>>=</code>	<code>&&</code>	<code> </code>	<code>!</code>
Python	同じ	<code>and</code>	<code>or</code>	<code>not</code>

繰り返し — `for` と `range`

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):       # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)

for i in range(5, 0, -1):   # 5, 4, 3, 2, 1
    print(i)
```

- `range(終了)`, `range(開始, 終了)`, `range(開始, 終了, 刻み)`
- 終了の値は含まない (C の `i < n` と同じ感覚)

for は「並びを順に取り出す」構文

C の `for (int i = 0; i < 5; i++)` に相当するが、Python の `for` は並びの要素を順に取り出す構文:

```
for c in 'abc':           # 文字列の各文字
    print(c)

for x in [10, 20, 30]:    # リストの各要素
    print(x)

for i, x in enumerate(['a', 'b']): # 番号付き
    print(i, x)           # 0 a / 1 b

total = 0
for i in range(1, 101):
    total += i            # total = total + i
print(total)             # 5050
```

繰り返し — `while`、`break`、`continue`

```
n = 1
while n < 100:
    n *= 2
print(n)          # 128

while True:        # 無限ループ
    s = input()
    if s == 'quit':
        break      # ループを抜ける
    if s == '':
        continue   # 次の周回へ
    print('echo:', s)
```

- C と同じ意味。 `do-while` は Python にはない

ネットワークプログラムでよく見る形

```
with socket.socket(...) as s:  
    s.connect((HOST, PORT))  
    while True:  
        data = s.recv(1024) # 受信  
        if not data:        # 相手が切断したら空  
            break
```

- サーバやクライアントは「接続が続く限り繰り返す」ので `while True` + `break` が定番
- 第 3 回以降、この形を何度も書く

リスト — 複数の値をまとめる

```
a = [10, 20, 30]          # リストを作る
print(a[0], a[-1])        # 10 30
print(len(a))             # 3

a.append(40)               # 末尾に追加 → [10, 20, 30, 40]
a[1] = 25                  # 書き換え   → [10, 25, 30, 40]
print(a[1:3])              # [25, 30]   スライス

b = []                    # 空リスト
for i in range(5):
    b.append(i * i)
print(b)                   # [0, 1, 4, 9, 16]
```

- C の配列に近いが、長さが可変で、異なる型を混ぜられる

リストのよく使う操作

```
nums = [3, 1, 4, 1, 5]
print(sum(nums), max(nums), min(nums))    # 14 5 1
print(sorted(nums))                       # [1, 1, 3, 4, 5]
print(3 in nums)                          # True
print(' '.join(['a', 'b']))               # 'a b'
```

入力を数値のリストにする

```
# 入力: 3 1 4 1 5
nums = list(map(int, input().split()))
print(nums)                             # [3, 1, 4, 1, 5]
```

- 詳しくは教材 `python_string-list.py` (第 2 回)

ここまでのまとめ — 最初の Python プログラム

「整数を 1 つ読み、1 からその数までの和を出力する」

```
n = int(input())
total = 0
for i in range(1, n + 1):
    total += i
print(total)
```

同じ内容を C で:

```
#include <stdio.h>
int main(void) {
    int n, total = 0;
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) total += i;
    printf("%d\n", total);
    return 0;
}
```

覚えておくこと

- `int(input())` で整数を 1 つ読む
- `range(1, n + 1)` で 1～n
- `print()` は自動で改行する
- インデントを揃える
- エラーメッセージは最後の行を読む
 - `NameError`: 変数名の打ち間違い
 - `TypeError`: 型が合わない (`str` + `int` など)
 - `IndentationError`: インデントがずれている
 - `SyntaxError`: `:` や `)` の付け忘れ

4. 演習課題の進め方

演習課題 ex1

課題 1: paiza ラーニング

1. 公開ページ「paiza ラーニングでクーポンコードを使う」に従い、学校フリーパスを登録
2. 講座 「Python 体験編 1: Python をはじめよう」（全 15 チャプター）を修了
3. 認定証の画面キャプチャを Teams の課題 ex01 に提出（ニックネームに学籍番号を含めること）

課題 2: aiJudge

1. 配布したアカウントで aiJudge にログイン
2. ex01 の問題ですべて満点を取る

aiJudge の使い方

- <https://judge.math.ryukoku.ac.jp/>
- 問題ごとに Python ファイル（`.py`）を提出
- 提出直後に自動採点。テストケースごとの結果と理由が表示される
- 満点になるまで何度でも提出可
- 採点は「標準入力 → 標準出力」の一致で判定
 - 余計な出力（プロンプトなど）は不正解になる
- 提出物は TA・教員が確認する。他人のコードを写さない

締切: 9/25（木） 24:00

次回予告と今日やること

次回（9/25）：計算機ネットワークの基礎、Python のデータ型

- インターネットの仕組み: IP アドレス、ポート、TCP/UDP
- Python の文字列・リスト・辞書
- 教材: `python_string-list.py`

今日の 5 講時（実習）

1. 事前課題 ex0 が済んでいない人は環境構築（TA に相談）
2. aiJudge にログインできることを確認
3. 教材ノートブック `python_tutorial.py` を一通り実行
4. paiza と aiJudge の課題に着手

質問は Teams 「★ 質問用チャネル」 か、教室の TA へ。